

Decision Doc: Automated RFQ to Quote Drafting

Problem 1, Fastlane AI Agent Engineer Assessment

What I built and why

I chose Problem 1, turning an incoming distributor RFQ into a drafted quote and reply email. It matches work I have already done. In my robotic process automation internship I built flows that pulled data out of incoming emails, cleaned it, and organized it for the business to use. I have also worked in inventory management, so matching items to a catalog and thinking about pricing and stock is familiar ground. The other option, a marketing content problem, was further from my background, so this was the clearer choice.

The hard part here is not writing text, it is judgment. The system has to match a plain sentence like "hydraulic hose, 2 wire braid, 3/4 inch" to the correct catalog part, and know when it should not guess. That idea shaped the whole build. The AI handles language, regular code handles anything involving money, and a person approves the quote before anything is sent out.

What I built and what I chose not to build

I built a Python pipeline using LangGraph to manage the steps in order. The AI, Claude, is used at exactly two points: reading the RFQ into structured line items, and writing the reply email. Everything in between, matching, pricing, and totals, is regular code that can be tested, because a system that could make a math error should never decide a price.

I did not use a no code tool like Make or n8n. Those tools are strong at connecting to inboxes and business systems, which this assessment does not need since those parts are mocked. The real work, a matching system with confidence levels and a reason for every decision, would have ended up as custom code hidden inside a no code tool anyway, with weaker testing. I would still use a no code tool for one thing in production: watching the inbox and starting the process.

I also chose not to let the AI make the catalog match itself. Its confidence score is not reliable, and its explanation is written after the fact rather than being the real reason for the choice. Its biggest risk is picking the wrong part number while sounding sure of it. A wrong or unsafe part is worse than a slow answer, so any match that is not certain goes to a person instead of being guessed.

How the system works

The system works in seven steps, with a stop in the middle where a person reviews the quote before anything goes out.

1. Read the file. The RFQ is a PDF or an email, turned into plain text.
2. Extract the details. The AI pulls out each line item, quantity, and product detail, with a confidence score and the exact original wording kept for review.
3. Match the catalog. An exact part number match is trusted right away. A part number not in the catalog is flagged, never guessed at. A line with no part number goes through a scoring system that checks size, pressure, material, and fittings, and anything under spec, like a hose rated for less pressure than requested, is rejected rather than treated as close enough.
4. Add pricing. Unit price, lead time, and totals come straight from the catalog using exact decimal math, never from the AI.
5. Put the quote together. Each line shows what the customer asked for, what was matched and why, and the price. The system also drafts a reply email, the second and last AI step, and the only number it may write there is the subtotal, which the code checks afterward.
6. Review. The process pauses and waits for a person, using a simple screen or the quote file directly, and will not continue while any item is unresolved.
7. Finalize. Once approved, the system recalculates every number from the final edited values, sends the reply email with the quote attached, and writes a record for the accounting system. Both are mocked for this assessment.

The review screen sits on top of the same functions used by the command line version, so the system runs fully from the terminal without it. I built the screen after the core steps were working, since a clear screen explains the reasoning to a non technical reviewer far better than a raw data file.

What happens when something goes wrong

If the AI returns something that does not match the expected structure, the system rejects it, tries once more with the error included, and stops with a clear message rather than passing bad data forward.

If the AI returns something that looks reasonable but is missing detail, the matching step catches it. During testing, one extraction dropped the word pneumatic from a line, and the result was a line flagged for review and left out of the total, not a wrong match. Low confidence results and tied matches always go to a person as a question, never a guess.

If a reviewer changes something, like a quantity, the system recalculates every number from the edited values, so an edited line can never keep an old price by mistake.

If the accounting system is down when a quote is approved, the quote stays marked as approved but not yet recorded, and the reply email is held until the record goes through, since an unrecorded quote is a bigger problem than a slightly delayed reply.

The review step is a real part of the process, not a formality. The reviewer sees the reasoning behind every line, can change any field, and must resolve every flagged item before approval. On a flagged line, the reviewer can accept the suggested match, choose a different part, mark it not currently available, or remove it. One unavailable item does not block the rest of the quote, and the reviewer can still reject the whole quote.

How it would connect to real business systems

Right now the system writes the accounting record to a file instead of sending it anywhere, since this is a prototype. In a real version, the reviewer approves the quote first, and nothing is sent to any outside system before that. The system would then log in to the accounting software with securely stored login details, check whether the customer exists and create them if not, then create the quote record with one line per approved item and a reference back to the original RFQ. Every write is set up so a retry cannot create a duplicate. If a write fails, the system retries automatically, and after repeated failures the quote is marked approved but not recorded, with a person notified and the email held until the record is confirmed. A real email would work the same way, with secure login and the same review step in front of it. For now, sending a real email is optional and off unless the reviewer turns it on.

What would carry over to other clients

The reusable part of this system is the overall shape: read in a document, use AI to pull out structured details, run those through a scoring system that only accepts confident matches, build a priced quote with a reason for every line, pause for a person to review, recalculate on any edit, then send the result. What changes for each new client is smaller: the product catalog and its scoring weights, the terms that client's customers use, the wording the AI looks for, and how the final record maps into that client's systems. To check this held up outside the sample data, I wrote one more RFQ myself in a format the system had never seen, and it worked correctly end to end, including catching a stock shortage and a tight delivery deadline that none of the four original samples even included.

What is left to do next

With more time, I would connect a real inbox so new RFQs start the process automatically, build the real accounting system connection behind the same approval step, and let the matching system improve from corrections reviewers make over time. I would also finish support for scanned RFQs, which is partly built but not yet tested. As the catalog grows much larger, the matching step would need a faster way to narrow down candidates before scoring them, without changing how the scoring itself works. I would also turn the current test checks into a larger set of tests based on real past RFQs and their approved quotes.